## 6. VACUUM CLEANER PROBLEM

### AIM

To write a Python program that simulates a simple vacuum cleaner agent to clean two rooms.

### ALGORITHM

1. Initialize rooms A and B as Dirty and place the vacuum cleaner in room A.
2. Repeat while any room is Dirty.
3. If the current room is Dirty, clean it.
4. Otherwise, move the vacuum cleaner to the other room.
5. Display the final status of both rooms.

### PROGRAM

```
rooms = {'A': 'Dirty', 'B': 'Dirty'}
position = 'A'

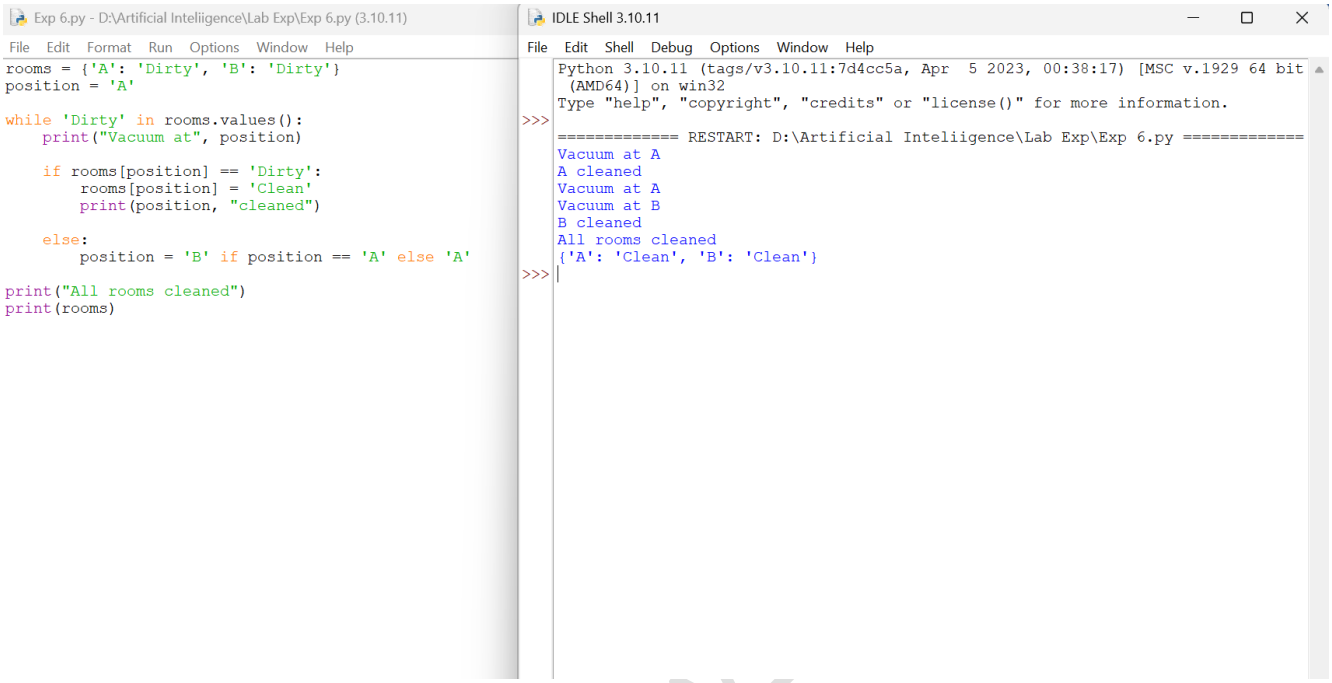
while 'Dirty' in rooms.values():
    print("Vacuum at", position)

    if rooms[position] == 'Dirty':
        rooms[position] = 'Clean'
        print(position, "cleaned")
    else:
        position = 'B' if position == 'A' else 'A'

print("All rooms cleaned")
print(rooms)
```

Thirulogasundar Y- 19232107

## OUTPUT




## RESULT

Thus, the Python program successfully simulates the vacuum cleaner agent and cleans both rooms.

Thirulogasundar Y

## 7. BREADTH-FIRST SEARCH (BFS)

### AIM

To implement Breadth-First Search (BFS) in Python to traverse a graph level by level.

### ALGORITHM

6. Represent the graph using an adjacency list.
7. Create an empty visited set and a queue containing the starting node.
8. While the queue is not empty, remove the front node.
9. If the node has not been visited, mark it visited and print it.
10. Add all its unvisited neighbours to the queue.
11. Stop when the queue becomes empty.

### PROGRAM

```
from collections import deque

graph = {
   'A': ['B', 'C'],
   'B': ['D', 'E'],
   'C': ['F'],
   'D': [],
   'E': [],
   'F': []
}

def bfs(start):
   visited = set()
   q = deque([start])

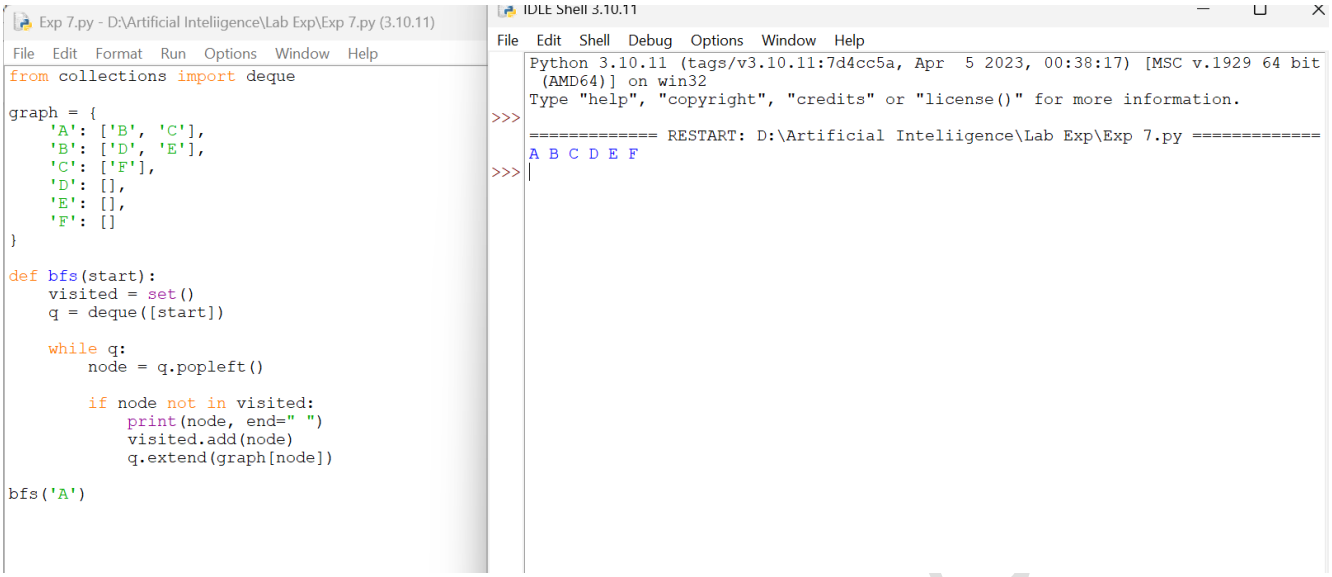
   while q:
      node = q.popleft()

      if node not in visited:
         print(node, end=" ")
         visited.add(node)
         q.extend(graph[node])

bfs('A')
```

Thirulogasundar Y- 19232107

## OUTPUT




## RESULT

Thus, the Python program successfully performs Breadth-First Search and prints the graph nodes in breadth-first order.

Thirulogasundar Y- 192

## 8. DEPTH-FIRST SEARCH (DFS)

### AIM

To implement Depth-First Search (DFS) in Python to traverse a graph by exploring each branch as deeply as possible.

### ALGORITHM

12. Represent the graph using an adjacency list.
13. Create an empty visited set.
14. Start the DFS procedure from the given starting node.
15. If the node has not been visited, mark and print it.
16. Recursively call DFS for each unvisited neighbour.
17. Continue until all reachable nodes have been visited.

### PROGRAM

```
graph = {
   'A': ['B', 'C'],
   'B': ['D', 'E'],
   'C': ['F'],
   'D': [],
   'E': [],
   'F': []
}

visited = set()

def dfs(node):
   if node not in visited:
      print(node, end=" ")
      visited.add(node)

      for neighbour in graph[node]:
        dfs(neighbour)

dfs('A')
```

Thirugasundar Y- 19232107

## OUTPUT




## RESULT

Thus, the Python program successfully performs Depth-First Search and prints the graph nodes in depth-first order.

Thirulogasundar Y-192

## 9. TRAVELLING SALESMAN PROBLEM (TSP)

### AIM

To write a Python program to find the minimum-cost tour that visits every city exactly once and returns to the starting city.

### ALGORITHM

18. Represent the distances between cities using a cost matrix.
19. Fix city 0 as the starting city.
20. Generate all permutations of the remaining cities.
21. For each permutation, form a tour that starts and ends at city 0.
22. Calculate the total cost of the tour.
23. Keep track of the tour with the minimum cost.
24. Display the best tour and its minimum cost.

### PROGRAM

```
from itertools import permutations

graph = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]

n = len(graph)
cities = range(1, n)
min_cost = float('inf')
best_path = None

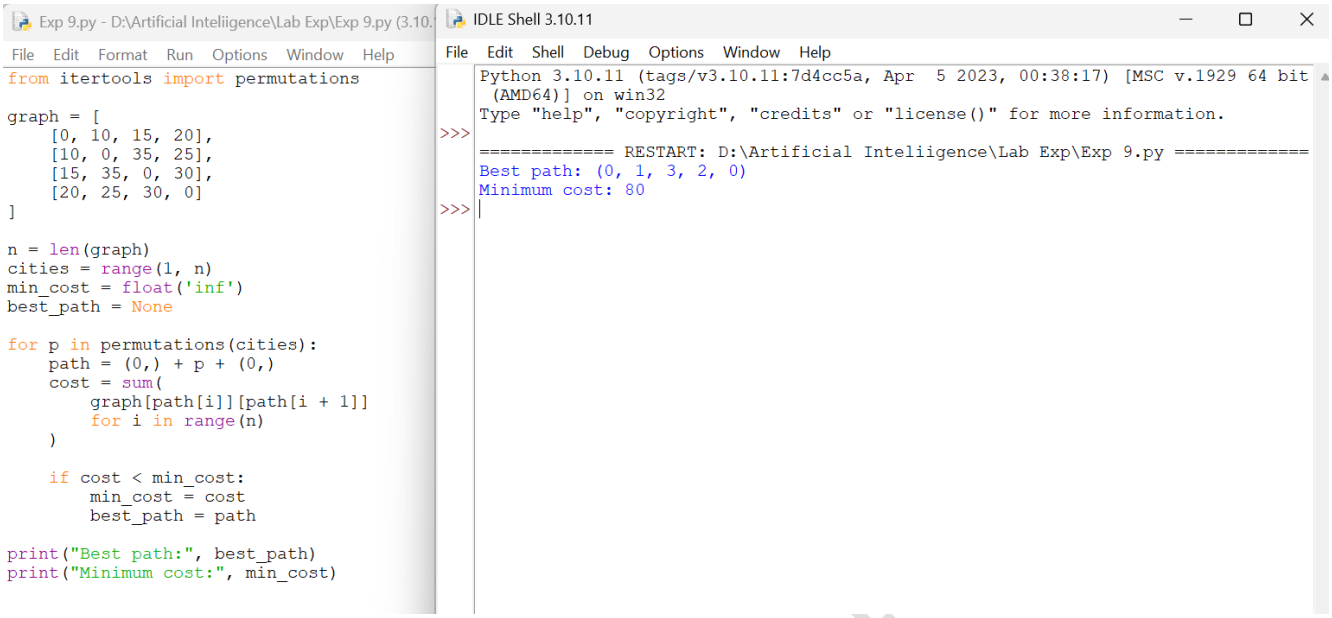
for p in permutations(cities):
    path = (0,) + p + (0,)
    cost = sum(
        graph[path[i]][path[i + 1]]
        for i in range(n)
    )

    if cost < min_cost:
        min_cost = cost
        best_path = path
```

Thirulogasundar Y- 19232107

print("Best path:", best_path)
print("Minimum cost:", min_cost)

**OUTPUT**




**RESULT**

Thus, the Python program successfully finds a minimum-cost travelling salesperson tour. The minimum tour cost for the given matrix is 80.

Thirulogasundar Y - 2

## 10. A* SEARCH ALGORITHM

### AIM

To implement the A* search algorithm in Python to find a path from a start node to a goal node using path cost and heuristic estimates.

### ALGORITHM

25. Represent the graph with edge costs and assign a heuristic value to each node.
26. Insert the start node into a priority queue with priority f(n) = g(n) + h(n).
27. Remove the node with the lowest priority from the queue.
28. If it is the goal node, display the path and total cost.
29. Skip the node if it has already been visited.
30. For each unvisited neighbour, calculate the new path cost and its f-score.
31. Insert the neighbour and updated path into the priority queue.
32. Repeat until the goal is found or the queue becomes empty.

### PROGRAM

```
import heapq

graph = {
   'A': [('B', 1), ('C', 3)],
   'B': [('D', 3), ('E', 1)],
   'C': [('F', 5)],
   'D': [],
   'E': [('F', 2)],
   'F': []
}

h = {'A': 5, 'B': 4, 'C': 4,
    'D': 3, 'E': 1, 'F': 0}

def astar(start, goal):
   pq = [(h[start], 0, start, [start])]
   visited = set()

   while pq:
      f, cost, node, path = heapq.heappop(pq)

      if node == goal:
         print("Path:", path)
         print("Cost:", cost)
```

Thirulogasundar Y- 19232107

```
        return

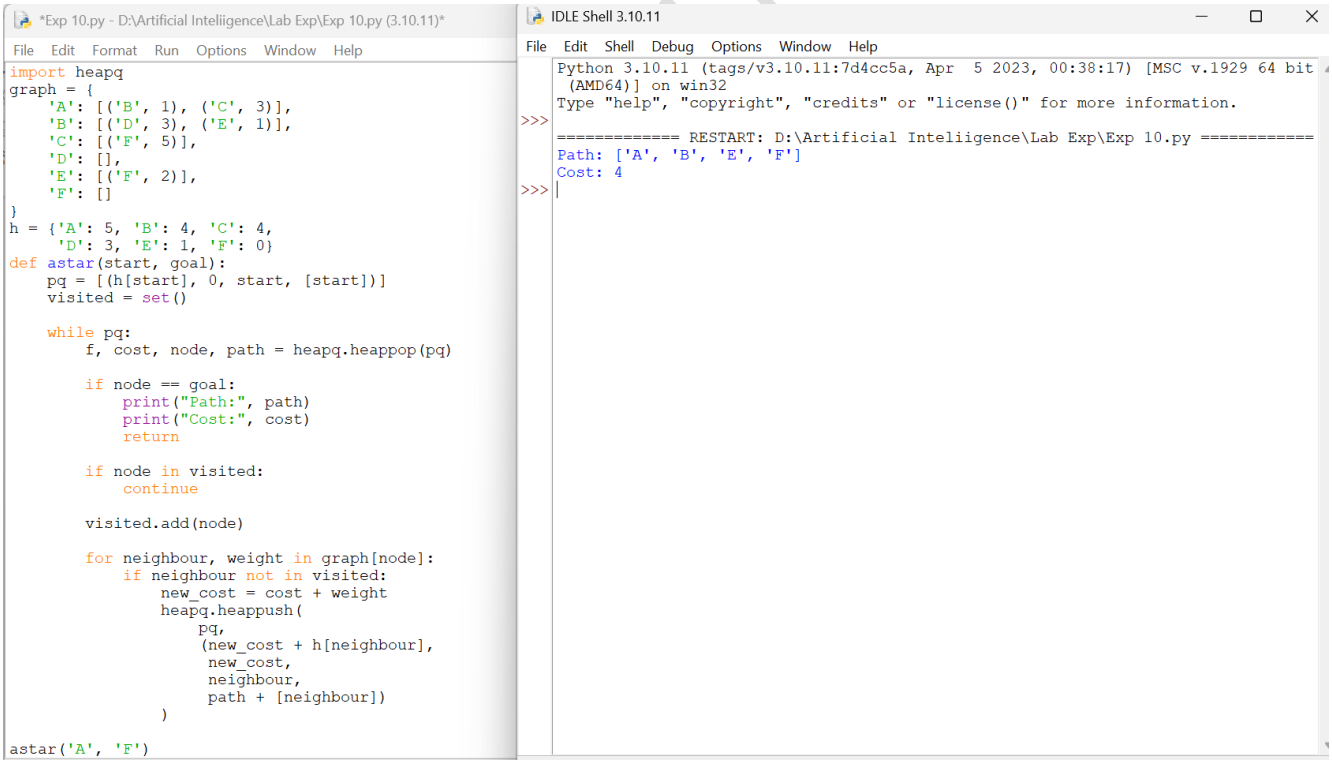
    if node in visited:
        continue

    visited.add(node)

    for neighbour, weight in graph[node]:
        if neighbour not in visited:
            new_cost = cost + weight
            heapq.heappush(
                pq,
                (new_cost + h[neighbour],
                 new_cost,
                 neighbour,
                 path + [neighbour])
            )
```

astar('A', 'F')

**OUTPUT**




**RESULT**

Thus, the Python program successfully implements A* search and finds the path A → B → E → F with a total cost of 4 for the given graph.

Y-19232073